

Decoding Lecture 1 - EECE 571-Q

Kjell Moller, Arman Lotfalikhani

July 2026

1 Introduction

Current quantum hardware is noisy and susceptible to a wide range of physical errors that serve as a barrier for quantum computation. Because of this problem, Quantum Error Correction (QEC) is a primary focus of modern research, with the goal of developing a Fault-Tolerant quantum computer. However to achieve truly fault tolerant computation we need to find methods of suppressing physical error rate during Circuit-Level noise [2] to a point where computation is reliable. To overcome physical noise, we encode physical qubits as logical codes. Where we can then stabilize our states to be tolerant to various sets of errors. However when an error does occur, we need a method of tracing back to and applying a recovery operation to ensure that our logical state is unaffected by errors on its physical qubits.

The motivation behind this lecture is exploring the mathematic objective to this problem and the solutions currently devised to solve it. We will walk through how we can map a measured error syndrome into a recovery operation for our logical state, as well as how a decoder needs to fit into the overall QEC pipeline and the constraints it must operate within.

1.1 Background

Given a quantum code C , we can stabilize the code space with the stabilizer of the code S . Since all stabilizers commute with each other, and leave the code state unchanged, if an error is introduced we can identify the syndrome as $He = s(mod 2)$. The syndrome is representative of which stabilizers anticommute with the error, allowing us to identify which nodes. However, multiple physical errors may have the same syndrome in a degenerate code.

With a given stabilizer S , if a Pauli error is contained in the normalizer but not in the stabilizer, this will cause a logical error on the state as it commutes with all of the stabilizers but will induce a logical change:

$$e \in N(S) \setminus S$$

If an error is contained in the stabilizer:

$$e \in S$$

It will produce no syndrome but will leave the code state unchanged. Finally if

$$e \notin N(S)$$

it will produce some syndrome that we need to identify a way to correct and recover from. This is a core mechanic of fault tolerance, where faults cannot spread into more errors than the code can correct. Knowing all of this, we need some method of identifying and recovering from errors that are not stabilized in the code.

1.2 Learning Outcomes

Up to this point in the course we have effectively learned the theory behind QEC and how to apply QEC codes to stabilize our logical states and lower our logical error rate. These learning outcomes aim to extend the students knowledge gained in the course by demonstrating how we can map our measured syndrome to an applied recovery operation. We provide students with numerous practical use cases and algorithms which were previously alluded to, so they can effectively implement a decoder and finalize how to build a QEC circuit from start to finish.

By the end of the lecture, students should be able to effectively:

- Formulate quantum decoding as an optimization problem over recovery operations

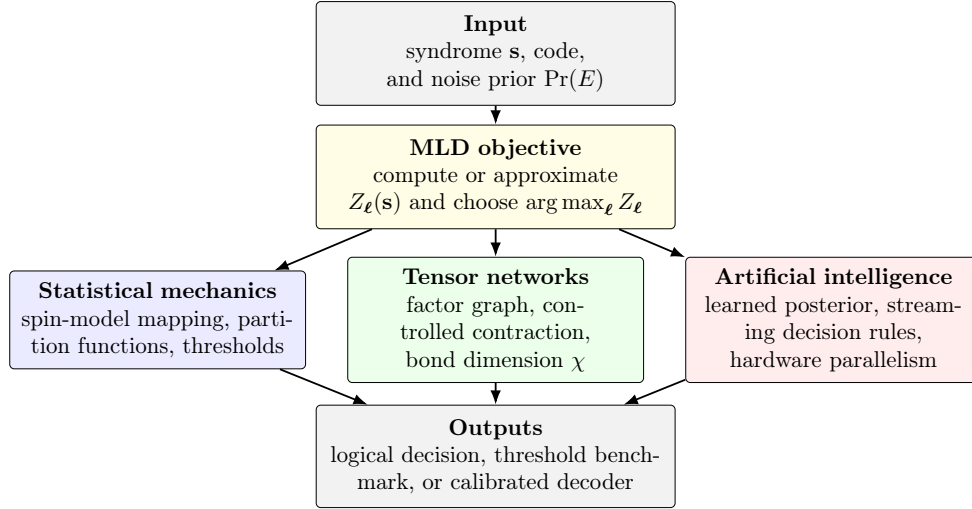


Figure 1: Classification of recent approaches to decoding. From: Figure 2 [2]

- Explain why maximum-likelihood decoding (MLD) is generally impractical and approximate decoders are needed
- Apply the algorithms of Belief Propagation (BP) and its variants, Minimum Weight Perfect Matching (MWPM) and Union-Find (UF), from syndrome to recovery
- Identify common performance metrics in QEC decoding, comparing algorithms and adapting to face the challenge of real-time decoding

2 The Decoding Objective

2.1 Maximum Likelihood Decoding

Let s be our observed syndrome on a parity check matrix H . Where $He = s$ defines our stabilizer measurement. The goal of a decoder is to map an observed syndrome to a recovery operation $s \rightarrow R$. Each decoders objective is to apply a recovery to ensure that when combined with our observed syndrome our logical state is changed. The decoding objective can be defined as [2]:

$$L^* = \underset{L}{\operatorname{argmax}} \{Pr(L|s)\}$$

Where L is the logical class. In reality this problem is intractable. As we scale up the distance of our codes we end up summing over an exponential amount of states consistent with our observed syndrome. Which is not a feasible problem to be solved in practice.

2.2 Decoder Families

Given that MLD cannot be solved efficiently, some method needs to be devised to approximate the objective function. This is where surrogate functions come into play.

Figure 1 proposes 3 (4) different families of decoders that encompass current research. Statistical mechanics, tensor network, and artificial intelligence based decoders (along with classical decoders). Each one of these families, and decoders within them will have some other mathematical objective function that approximates the decoding objective function. Often times classical algorithms are implemented as functional baselines for research on other decoder families, but can perform well outright on their own.

2.3 Tanner Graphs and Geometrical Properties

The factor graph of an $[n, k, d]$ classical code is a bipartite graph $G = (V, U, \Lambda)$ with an adjacency matrix given by the code's parity check matrix H . [13]

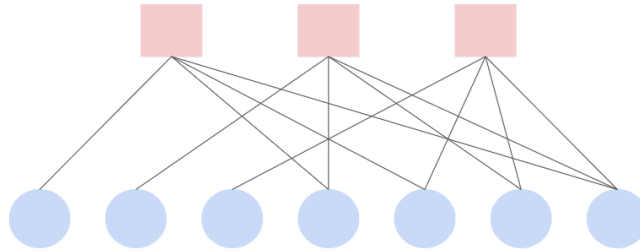


Figure 2: Tanner Graph of the $[n,k,d] = [7,4,3]$ Hamming Code. With check nodes in red and variable nodes in blue.

Tanner graphs allow us to easily visualize our codes stabilizer and qubit relationships by defining our data nodes or variable nodes as the columns of our parity check matrix H , and stabilizers or check nodes as our rows of the matrix. A graph edge is drawn between each node where the corresponding row and column values $H_{i,j} = 1$.

With our visualization, we are able to easily identify how errors can propagate through our graph and . The density of edges, formally known as the sparsity directly impacts how a given syndrome applied to a check node can impact many data qubits at once.

An (lQ, qQ) -QLDPC code family is defined as a set of CSS codes whose quantum parity check matrices $HCSS$ have row and column weights upper bounded by lQ and qQ respectively. [3, 12]

QLDPC codes are a primary focus of modern QEC as they prevent the edge weight from increasing to a level that hinders computation. Surface codes, Hamming codes, and the many of the other codes we have seen in class fall under this umbrella.

Since our edge weight is bounded on QLDPC codes, their Tanner graphs have geometrical properties that can be exploited. As we will see in the coming sections, knowing that there is a maximum edge weight can lead to limitations on how errors can appear on our check nodes, allowing us to find surrogate functions for our decoding objective and find better time complexity for decoding.

3 Classical Decoding

This section explores conventional decoders that are used in current practice as baselines for modern decoder architectures.

3.1 Union-Find

For topological codes, errors exhibit a pairlike structure in their tanner graphs. As they are CSS codes, we can isolate by X and Z type stabilizers to obtain a tanner graph where each variable node connects to two check nodes of each stabilizer type. To set up Union Find on an unrotated surface code, first the graph is transformed into a similar structure where check nodes are represented by a node on the graph, and variable nodes as edges between. We then define a boundary on the left and right edges of the graph where the outermost edges connect to. Quiz Question 2 in Figure 9. (a) visually outlines the setup of UF.

The algorithm begins with identified syndrome defects on check nodes, where a cluster $C_i = v_s$ is defined at each node with a syndrome defect. The algorithm then expands each cluster by half an edge each step, according to the following rules [4, 18]:

1. If two clusters touch, merge the two clusters into one larger cluster
2. If the parity of a cluster's syndrome defects is even, terminate the cluster's growth
3. If a cluster touches a border, terminate the cluster's growth
4. If the parity of a cluster's syndrome is odd, continue growing in the next step

A good video that was drawn as inspiration for quiz questions [16] illustrates the growth of these clusters as well as the relation to the following algorithm covered, Minimum Weight Perfect Matching (MWPM). Once all clusters have terminated their growth, each syndrome defect should be paired within each cluster. To accomplish this, we

apply the peeling algorithm [5]. This algorithm allows us to find an error chain connecting each defect. We first initialize a spanning forest between each disconnected cluster of its vertices and edges:

$$F_\epsilon = (V_F, E_F)$$

We then apply the identified syndrome to check nodes, where $s(v) = 1$ if an error is identified and $s(v) = 0$ otherwise. We then check each leaf of F_ϵ and apply the following rules:

1. If $s(v) = 1$: we include the edge between vertex v and its parent u $e = (u, v)$ in the error chain ϵ_c
2. If $s(v) = 0$: we do not include V in the error chain ϵ_c
3. Remove the leaf V from F_ϵ

This process terminates once there are no more vertices in the spanning tree. After which we should have some matching path between each syndrome defect. One intuitive video explaining the algorithm is [15]. Peeling will always produce a valid matching between syndrome defects within clusters, but it will not necessarily always be minimum weight [9]. This motivates the next algorithm that we will cover.

3.2 Minimum Weight Perfect Matching

Union-Find's issue with finding a minimum weight matching is that it does not preserve internal cluster information. Any defect can be paired with any other defect within a cluster so long as it produces a valid, not necessarily optimal, recovery. [8] The Blossom Algorithm is a method devised to retain this internal cluster information. MWPM implements the blossom algorithm's dual variable tracking method for cluster sizes [18]. For our graph defined as above with UF with variable nodes as edges E and check nodes as vertices V . The edge weight w_e is defined as the cost of the solution error path. The primal objective function is defined as:

$$\min_x \sum_{e \in E} w_e x_e$$

Subject to the constraint of

$$\sum_{e \in v} x_e = 1$$

This constraint implies that each vertex with a defect is paired with only one edge part of an error chain. And the objective allows us to minimize the distance of the error chain. The dual objective is defined as:

$$\max_y \sum_{s \in O} y_s$$

Subject to the constraint of

$$\sum_{s \in O} y_s \leq w_e$$

Intuitively this is the amount that each region has grown from it's origin. The edge slack can be defined as:

$$slack(e) = w_e - (y_u + y_v)$$

A tight edge occurs when $slack(e) = 0$, which occurs when the blossoms of each cluster touch.

With these definitions in mind, the rules for MWPM state:

1. Grow our dual variables y , which define the growth of each region, outward at each step
2. If slack becomes zero between two dual variables. We consider the following options
3. If the dual cluster being touched is unmatched, we match the two defects and terminate growth
4. If the dual cluster being touched already has a valid matching (even parity). We engage in alternating growth between the matched cluster to allow the existing cluster to continue growth through the existing matching. This effectively means that the closer dual variable to the collision will shrink, and another will grow to permit the other cluster to grow through (see [16]). If alternating growth does not find another matching, another blossom will form around all clusters

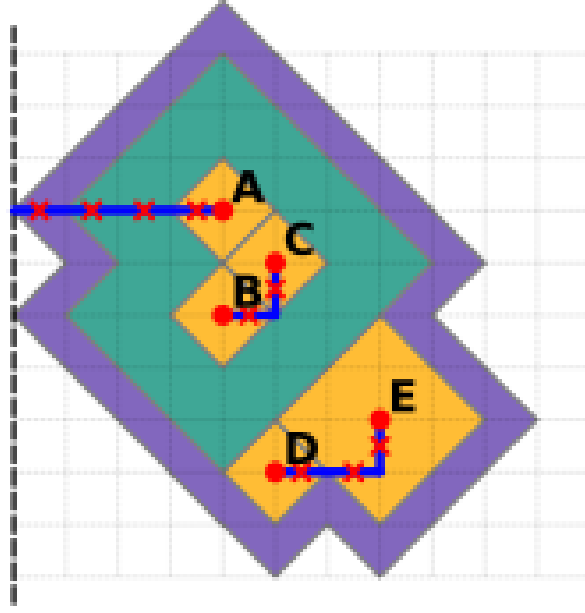


Figure 3: Figure 7, (8) from [18], illustrating the blossom algorithm on a surface code syndrome

5. If the dual cluster being touched forms an odd group of clusters, we fuse these into a larger vertex and begin a blossom, which is a new dual variable search region containing the growth of all of the clusters
6. We then grow each blossom outwards until all clusters/blossoms terminate with pair matchings minimizing the edge weight and maximizing our dual variables

This algorithm ensures that defects can only be matched with other defect contained within the same blossom. Circumventing the problem that UF had with a lack of internal cluster information. The annotated lecture slides contain an illustration of the blossom algorithm. To accompany this Figure 7 of [18] has a strong visualization of the different steps in the algorithm.

When comparing the two algorithms, Union-Find performs the same as MWPM under conditions of cluster touching both boundaries, and UF clusters having the same shape as MWPM clusters in one solution [16]. Some current improvements over the original blossom algorithm include sparse blossom [9], and fusion blossom [17], which parallelizes the blossom algorithm to achieve better performance.

3.3 Belief Propagation

While MWPM and Union-Find function well for the pair like structure of surface codes, they do not extend well to codes without this structure. For a more universal decoding algorithm, Belief Propagation extends much better to a broader family of codes.

Belief propagation functions by going through a series of cycles on the Tanner graph, where based on the syndrome it will update the probability or "beliefs" that certain variable nodes had an error occur. At each iteration it will check if any nodes have a belief greater than 0.5, and make a hard decision of whether that node had an error. BP follows the following sequence of operations:

1. Initialize each variable node with the physical error probability p_e
2. Send messages from variable to check nodes, where $n(c) \setminus q$ is all edges connected to c besides q , defined as (Equation 9 [10]):

$$m_{q \rightarrow c}(E_q) \propto p(E_q) \prod_{c' \in n(q) \setminus c} m_{c' \rightarrow q}(E_q)$$

Meaning our message is proportional to taking a product of all of our prior messages that did not come from the check node being sent to, and multiply it by our current belief

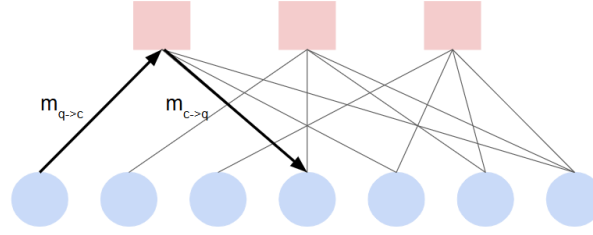


Figure 4: Belief Propagation Message transfer on the Hamming Code

- Return messages from check to variable nodes given their new probabilities, defined as (Equation 8 [10]):

$$m_{c \rightarrow q}(E_q) \propto \sum_{\substack{E_{q'} \\ q' \in n(c) \setminus q}} (\delta_{S_c, S_c \cdot E_c} \prod_{q' \in n(c) \setminus q} m_{q' \rightarrow c}(E_{q'}))$$

Meaning our message is proportional to summing all of our products of prior messages from variable to check nodes multiplied by the current syndrome

- Update the probabilities of each variable node given the new information (Equation 10 [10]):

$$p_q(E_q) = p_{q \text{ prior}}(E_q) \prod_{c \in n(q)} m_{c \rightarrow q}(E_q)$$

Here the new belief probability is the prior belief multiplied by the product of all other check to variable messages

- Check each variable node i and make a hard decision on whether $0.5 \leq p_i$
- If a hard decision was made on $\hat{e}$, double check whether $H\hat{e} = s$. If so, terminate. If not, continue iterations

However, Belief Propagation has a few fundamental flaws, which are the short cycle and degeneracy. A short cycle is a series of connected edges in the Tanner graph where through following a series of edges you will trace back to the same node. Short cycles and degeneracy can cause information to spin in BP and as a result will make it converge much slower or not at all. As such, BP needs some other algorithm or decision to overcome this.

3.3.1 Ordered Statistics Decoding

To circumvent these issues with BP, one option is to use Ordered Statistics Decoding (OSD). OSD produces a syndrome consistent correction if the hard decision of BP does not match the syndrome. Once BP is finalized, it provides us with the probability of each variable node having an error. Using these, the steps of OSD-0 are as follows [12]:

- Reorder the columns of H in order of each columns BP error probability, ordered from the highest values on the left to lowest on the right of the new matrix to obtain H_{OBP}
- Partition the new H_{OBP} into a linearly independent set of $rank(H)$ variables as a set S , and the remainder set T
- Solve for S where

$$e_{[S]} = H_{[S]}^{-1} \cdot s$$

. The inverse matrix here indicates a solved matrix using Gaussian Elimination

- Set the remaining columns to 0 and assume that the correction is contained in the most likely set S given by BP

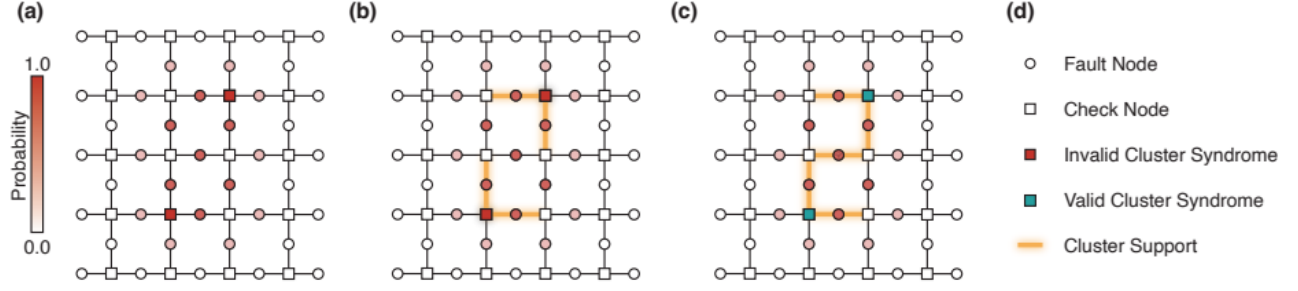


Figure 5: Figure 2. of [7] Localized statistics decoding reliability-based weighted cluster growth for the surface code

Higher order OSD consists of testing configurations not set to 0 for the dependent variable set T as we consider (Equation 8 [12]):

$$e_{[S,T]} = (H_{[S]}^{-1} \cdot e_{[S]} + H_{[S]}^{-1} \cdot H_{[T]} \cdot e_{[T]}, e_{[T]})$$

Where we sweep across possibilities of $e_{[T]}$ according to the weights.

In the case of OSD-0 this simplifies to

$$e_{[S,T]} = (e_{[S]}, e_{[T]}) = (e_{[S]}, 0)$$

OSD's main issue is that this matrix solve needs to run globally, meaning that at each decoding cycle with OSD there is a large computational overhead of the reliability sorting and matrix solve. Especially with larger codes, this can dominate the overall decoders time complexity, which is non ideal as we will discuss in section 4.

3.3.2 Localized Statistics Decoding

Since OSD suffers from the performance of having to do a global matrix solve on each decoding cycle, which does not scale well. [7] proposes a variant of OSD where instead of a global matrix solve we isolate to local clusters. This reduces the computation required significantly and also gives the ability to parallelize the algorithm through on the fly elimination on smaller clusters, in a similar way to fusion blossom [17].

Rather than solving the global decoding equation $He = s$ directly, we initialize a cluster C_i beginning at each measured syndrome defect. After BP completes, we expand the cluster to include a neighboring node with the highest BP probability. Each cluster parity check matrix H_{C_i} will then be expanded to include these new nodes, as visualized in Figure 5. *On the fly elimination* involves applying a Gaussian Elimination step at each cluster expansion, where we only need to eliminate the added column rather than an entire matrix solve to check if our local solve can satisfy the equation

$$H_{C_i} e_{C_i} = s_{C_i}$$

If this solve is not successful, continue adding the next most likely node on the clusters locality to find the our next addition. This repeats until we satisfy our syndrome equation. In the case of two clusters colliding, we merge their respective matrices and form one larger cluster that we will continue applying on the fly elimination to and expanding until we satisfy the combined syndrome equation. Once every cluster has completed we merge their corrections as a union to find our final solved $\hat{e}$.

By partitioning our clusters we are able to parallelize our computing resources to more efficiently solve for our syndrome on our code. Not just this, but it also limits the size of the matrix solve of each Gaussian Elimination step, as only in the worst case will our clusters expand to cover the entire code and effectively become an OSD solve.

4 Real-Time Decoding

4.1 The Backlog Problem

Practical decoder implementations are not just limited by their scalability and ability to suppress errors, but also by their runtime. [14] proposes a recursive backlog constraint, where if our decoder implementation is not able to keep up with the throughput of syndrome generation by the hardware, we will encounter an exponential growth in

the backlog of syndromes waiting to be decoded. If we have a queuing time from our generated syndromes t_q and processing time of the decoder t_p , we can define the ratio between these as:

$$f = \frac{t_p}{t_q}$$

At each decoding cycle D_k we can define the time taken to decode if D_k depends on the measurement of D_{k-1} as:

$$D_k = f D_{k-1}$$

As we expand our number of decoding cycles n , we can see that if $f > 1$ we will see an exponential growth in decoding time of $D_n = f^n D_0$. In light of this, our decoder implementations need to have an as good or better throughput than our hardware to avoid this exponential queuing backlog.

Modern systems require an estimated syndrome decoding cycle time of approximately $1\mu s$ for superconducting transmon qubits and $1ms$ for trapped ion qubits [1]. This motivates research on not just finding strong decoders which can effectively minimize our logical error rate, but also aiming for these decoders to operate under the constraints described for the relevant hardware.

4.2 Detector Error Models

Detector Error Models (DEM) are an abstraction enabling efficient and methodical translation of our circuit level representations into a node representation easily usable by our decoders. They enable us to compile our Clifford circuits down to a matrix representation similar to a Tanner Graph of the code and stabilizers. This detector error matrix contains information on which errors violate which detectors, as well as the associated probability of each error occurring, and whether they will flip a logical observable on our code [6].

Within the matrix, each detector is formed by comparing stabilizer measurements between consecutive rounds:

$$d_1 = m_{1,t} \oplus m_{1,t-1}$$

This allows it to not just observe stabilizer measurements but be able to differentiate between rounds and decide whether there is a difference in the new measurement parity. Similarly we can then define our error matrix Ω where $\Omega_{i,j}$ contains our measurement outcomes in rows and fault mechanisms in columns (think of them as locations where errors may occur and propagate). The matrix is indexed with 1 where a column error would propagate to that row's measurement. Our DEM matrix then can be defined as

$$H = D\Omega$$

The DEM effectively functions as a bridge between our decoder and the circuit, as the decoder only cares about the measured syndrome associated with the code being employed.

$$\text{Circuit} \rightarrow \text{DEM} \rightarrow \text{Decoder}$$

This is very useful for simulation, allowing us to fire detectors with events and analyzing how our decoder responds to errors. We only need to compile the circuit once into the model, which is then able to be rapidly interpreted by the decoder.

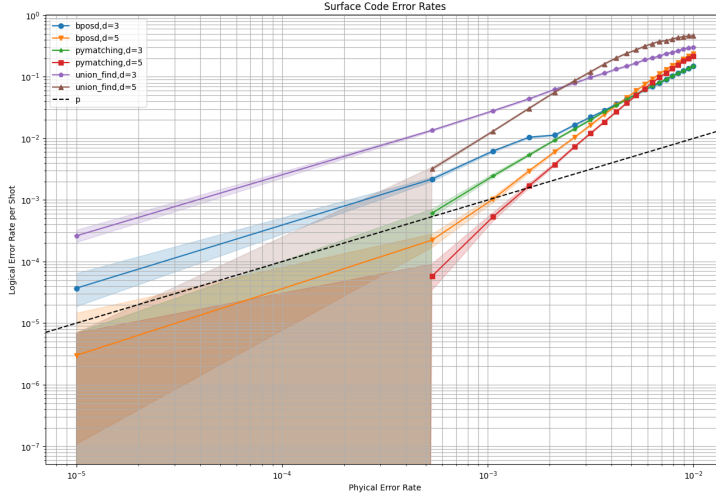
4.3 Comparing Decoders

Different use cases will have different requirements around decoder implementations. One scenario may involve needing to be a live real time system, where although accuracy is ideally high the system must run quickly enough to avoid the *backlog problem*. This contrasts with a case where a circuit is being run in a simulation to test accuracy where runtime is not as imperative.

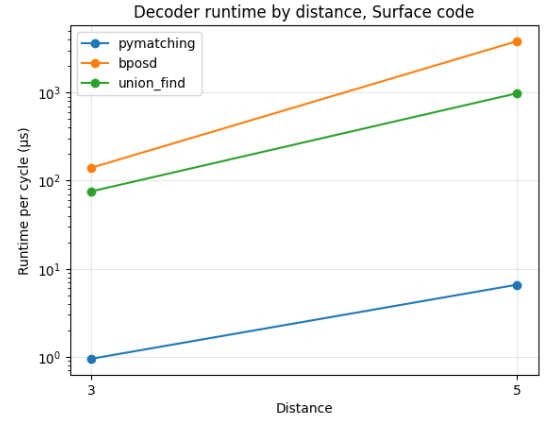
Some of the defining metrics used to compare decoders include:

- Error threshold p_{th} , the physical error rate below which scaling our code distance will present a decrease in logical error rate
- Decoding runtime T_d per decoding cycle
- Logical error rate p_l , compared across distances

Testing these metrics in conjunction with analyzing the use case of the decoder will give insight into the best decoder for the situation. There is no outright "best" decoder, it is highly dependent on the geometry of the code in question and use case. This motivates the demonstration provided during the lecture, where various decoders were compared on rotated surface code and color code circuits to showcase differences in runtime and logical error rate suppression.

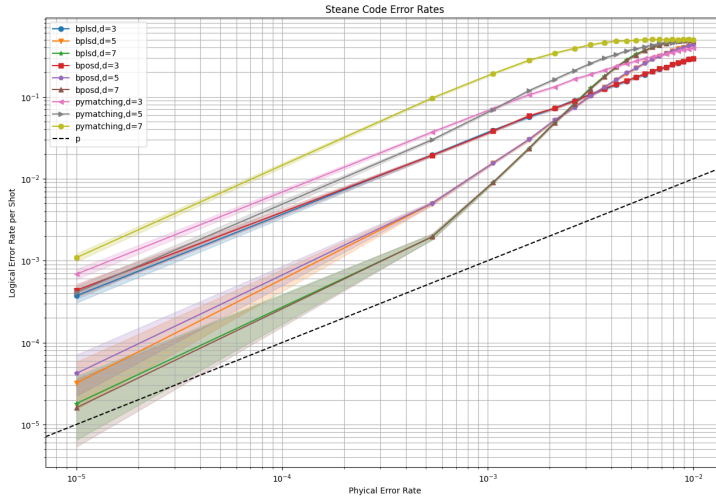


(a) Surface Code Error rate

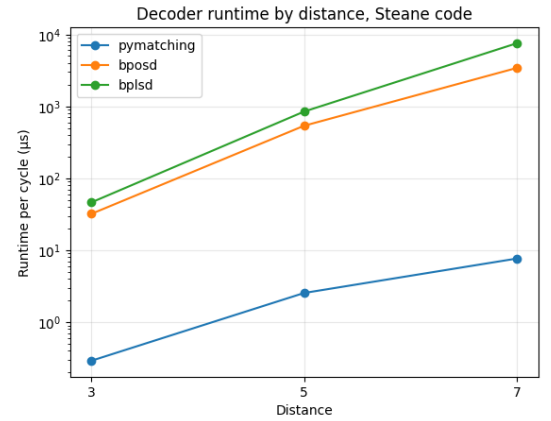


(b) Decoder runtime

Figure 6: Error rate and runtime results for the rotated surface code on Z stabilizers



(a) Steane Code Error rate



(b) Decoder runtime

Figure 7: Error rate and runtime results for the Steane code

Of note is that [11]'s Open Source LSD implementation is not yet parallelized, and so is expected to exhibit similar or potentially slightly worse performance than OSD given the context switching required to handle multiple clusters.

5 Supporting Material

5.1 Demonstrations

Here we present some figures from the demonstration presented in lecture. Decoders simulated include [11, 9] in the demonstrations. These demonstrations were simulated in Sinter using circuits built using Stim. Simulations were

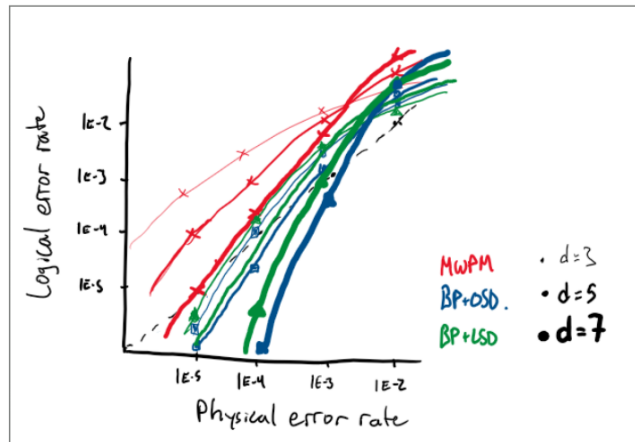
ran on a maximum of 1,000,000 shots and 10,000 errors per code distance on each code up to distances of 7. With greater computing resources this experiment could be extended to greater code distances.

Demo 1: Figures 6, 7 outline results from the first demonstration. This involved comparing various baseline decoders on multiple codes utilized in class using Stim and Sinter Clifford simulation. The goal of this demonstration was to show that decoders vary in performance across different codes, and also to illustrate how one can obtain results to compare decoders as described in section 4.3.

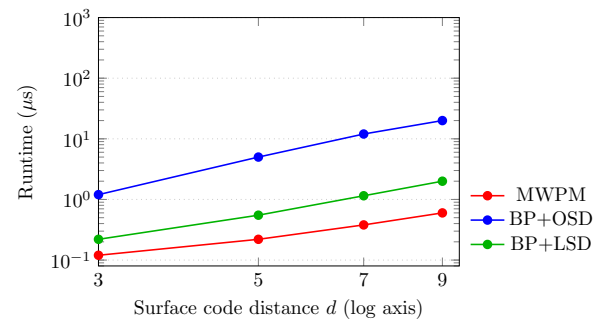
In the figures, most notable Pymatching performed best on the surface code by a significant margin, both in runtime and logical error rate suppression, with a much lower threshold value than BPOSD and UF. However, when transitioning to a Steane Code, the only codes that project as viable decoders at a higher distance are BPLSD and BPOSD, even though Pymatching shows a much better runtime. Once again the simulation was limited by computing resources, so at lower physical error rates the simulation could not be executed fully. Decoders must pass the $x = y$ barrier between the physical and logical error rate to be considered viable, as anything that introduces a higher logical than physical error rate is doing more harm than good. Many of the decoders in the demonstration do not cross this threshold, and so should not be considered as options for that specific code and experiment.

5.2 Quiz Questions

Quiz Question 1: a) An experiment requires a quantum decoder with strong logical error rate suppression. Runtime



(a) Physical vs. Logical Error rates for different decoders on the surface code.



(b) Decoder runtime

Figure 8: Quiz Question 1

matters but is not as important as accuracy and scalability since we are not running in real time. Which decoder is most appropriate?

Answer: BP+OSD Lowest error rate as well as shows most optimal trend in logical error rate scaling across distance.

b) A quantum device requires fast decoding that runs in real time on a trapped ion system. Accuracy is important but it needs to avoid an exponential growth in decoding time. Which decoder is most appropriate?

Answer: BP+LSD Both MWPM and BP+LSD are under the 1 microsecond threshold. So although MWPM is faster BP+LSD is a better choice since it does not have an exponential growth in decoding time and has higher accuracy.

Rationale: This question mainly targets learning objective 4 in lecture 1: Identify common performance metrics in QEC decoding, comparing algorithms and adapting to face the challenge of real-time decoding. Helping students make educated choices on the right decoder for the task while also testing the knowledge of RT decoding constraints

Quiz question 2: Syndrome defects are found at check nodes highlighted in red, with a corresponding number.

Apply the Union find algorithm and pair each syndrome together with another. If paired with a boundary use a capitalized B rather than a number. Express the pairs separated by commas, ordered with no spaces (ie. 12,34,56,78,9B. Please order in ascending order of sums, with B being worth 10, so that prairielearn doesn't get confused with answers.

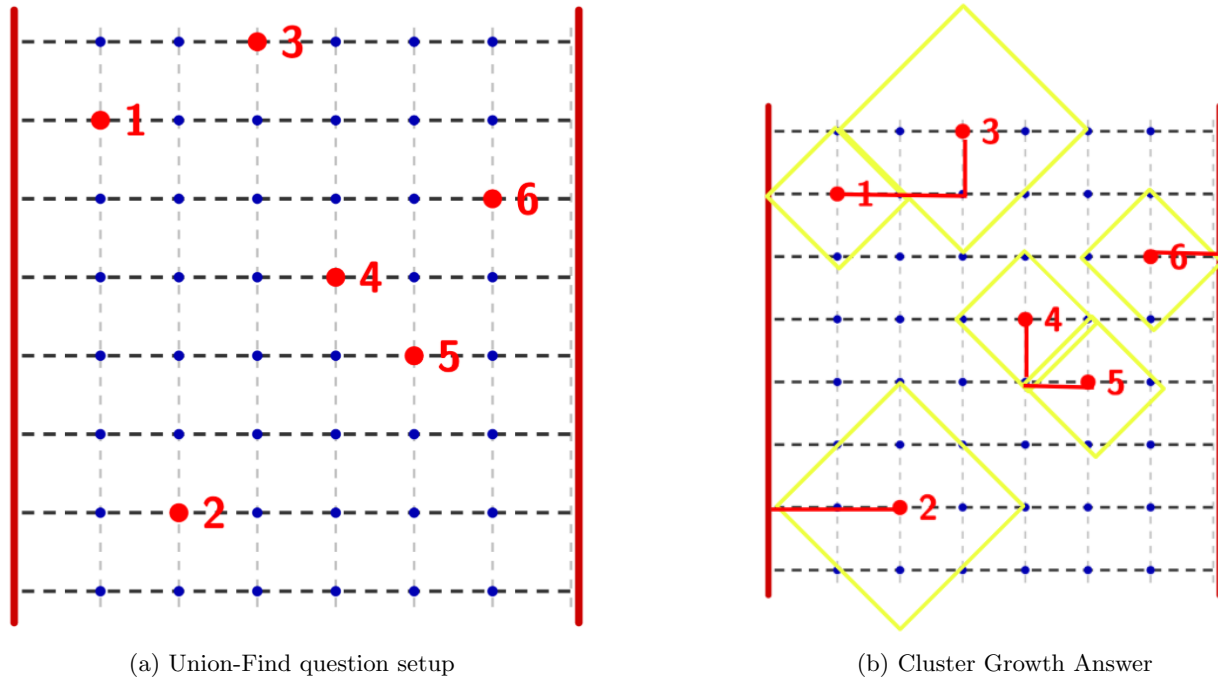


Figure 9: Quiz Question 2

Answer: 13,45,2B,6B Cluster growth is represented in Figure 9. (b).

Rationale: Inspired by [16], this question mainly targets learning objective 3 in lecture 1: Apply the algorithms of Belief Propagation (BP) and its variants, Minimum Weight Perfect Matching (MWPM) and Union-Find (UF), from syndrome to recovery. As Union Find is objectively the easiest to trace through of the four introduced in class by hand.

6 Conclusion

After completing this lecture, students should have a solid grasp of how a measured syndrome is translated to a recovery operation. They should also have some practical strategies and algorithms that they can understand and implement in UF, MWPM, and BP and its variants. They should also be able to identify scenarios and codes where each algorithm makes conceptual sense to implement and compare decoder performances through simulations and experiments.

7 Bibliography

References

- [1] F. Battistel et al. *Real-Time Decoding for Fault-Tolerant Quantum Computing: Progress, Challenges and Outlook*. 2023. arXiv: [2303.00054](https://arxiv.org/abs/2303.00054) [quant-ph]. URL: <https://arxiv.org/pdf/2303.00054>.
- [2] Hanyan Cao, Ge Yan, Yuxuan Du, and Feng Pan. *Maximum Likelihood Decoding of Quantum Error Correction Codes*. 2026. arXiv: [2605.17230](https://arxiv.org/abs/2605.17230) [quant-ph]. URL: <https://arxiv.org/abs/2605.17230>.
- [3] Paul L. McFadden David J.C. MacKay Graeme Mitchison. *Sparse Graph Codes for Quantum Error-Correction*. 2004. URL: <https://arxiv.org/pdf/quant-ph/0304161>.
- [4] Nicolas Delfosse and Naomi H Nickerson. “Almost-linear time decoding algorithm for topological codes”. In: *Quantum* 5 (2021), p. 595. URL: <https://arxiv.org/abs/1709.06218>.
- [5] Peter-Jan H. S. Derks, Alex Townsend-Teague, Ansgar G. Burchards, and Jens Eisert. *Designing fault-tolerant circuits using detector error models*. 2012. arXiv: [1703.01517](https://arxiv.org/abs/1703.01517) [quant-ph]. URL: <https://arxiv.org/pdf/1703.01517>.

- [6] Peter-Jan H. S. Derks, Alex Townsend-Teague, Ansgar G. Burchards, and Jens Eisert. *Designing fault-tolerant circuits using detector error models*. 2024. arXiv: [2407.13826 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/2407.13826>.
- [7] Timo Hillmann et al. *Localized statistics decoding for quantum low-density paritycheck codes*. 2025. arXiv: [2406.18655 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/2406.18655>.
- [8] Vladimir Kolmogorov. “Blossom V: A new implementation of a minimum cost perfect matching algorithm”. In: *Mathematical Programming Computation* (2009). URL: https://www.researchgate.net/publication/225752771_Blossom_V_a_new_implementation_of_a_minimum_cost_perfect_matching_algorithm.
- [9] Craig Gidney Oscar Higgott. *Sparse Blossom: correcting a million errors per core second with minimum-weight matching*. 2025. arXiv: [2303.15933 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/2303.15933>.
- [10] David Poulin and Yeojin Chung. “On the iterative decoding of sparse quantum codes”. In: *Quantum Information and Computation* (2008). arXiv: [0801.1241 \[quant-ph\]](#). URL: <https://arxiv.org/abs/0801.1241>.
- [11] Joschka Roffe. *LDPC: Python tools for low density parity check codes*. 2022. URL: <https://pypi.org/project/ldpc/>.
- [12] Joschka Roffe, David R. White, Simon Burton, and Earl T. Campbell. “Decoding Across the Quantum LDPC Code Landscape”. In: *Physical Review Research* (2020). arXiv: [2005.07016 \[quant-ph\]](#). URL: <https://arxiv.org/abs/2005.07016>.
- [13] R. Tanner. “A recursive approach to low complexity codes”. In: *IEEE Transactions on Information Theory* (1981). URL: <https://ieeexplore.ieee.org/document/1056404>.
- [14] Barbara M. Terhal. *Quantum Error Correction for Quantum Memories*. 2015. arXiv: [1302.3428 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/1302.3428>.
- [15] Simons Institute for the Theory of Computing. *Peeling Algorithms*. URL: <https://www.youtube.com/watch?v=uYAolk0btzo>.
- [16] Yue Wu. *APS2022 Interpretation of Union Find Decoder on Weighted Graphs and Application to XZZX Surface Code*. URL: <https://www.youtube.com/watch?v=BbhqUHKPdQk>.
- [17] Yue Wu and Lin Zhong. *Fusion Blossom: Fast MWPM Decoders for QEC*. 2023. arXiv: [2305.08307 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/2305.08307>.
- [18] Lin Zhong Yue Wu Namitha Liyanage. *An interpretation of Union-Find Decoder on Weighted Graphs*. 2022. arXiv: [2211.03288 \[quant-ph\]](#). URL: <https://arxiv.org/pdf/2211.03288>.